

Tool implementation — amounts.py

src/lab_billing/reconstruct/amounts.py — git blob 0e007fe2242d — v1.4.0 (2026-09-22)

```
"""Amount engine (module A, v1.0 — spec section 08).

Tracks gross posted, reversed, net observed posted and unallocated amounts.
Allocation sums exceeding the payment amount block the transaction's math
and record a conflict. Partial reversals are applied by their exact amount
and never double-deducted. Cross-currency amounts are never summed.

Attribution rules:
- A payment with allocations is split across the lineages of its allocated
  bills in proportion to the allocated amounts; the unallocated remainder is
  attributed to the payment's main lineage (its own bill, or the single
  allocated lineage, or UNLINKED when the allocation spans lineages).
- Reversal caps are enforced PER PAYMENT EVENT (a reversal can never deduct
  more than its own target payment), then distributed across the same
  attribution split as the gross amount.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any

@dataclass
class LineageAmounts:
    lineage_id: str
    gross_posted: int = 0
    reversed: int = 0
    net_observed_posted: int = 0
    unallocated: int = 0
    blocked_transactions: list[str] = field(default_factory=list)

@dataclass
class AmountConflict:
    kind: str
    detail: str
    event_ids: list[str]

class AmountEngine:
    def __init__(self, run_id: str):
        self.run_id = run_id
        self.conflicts: list[AmountConflict] = []

    def compute(self, events: list[dict], allocations: list[dict],
               lineage_of_record: dict[str, str]) -> dict[str, LineageAmounts]:
        """Returns lineage -> amounts. Only USD rows are counted; other
        currencies are reported as conflicts, never converted."""
        out: dict[str, LineageAmounts] = {}
        payment_amount: dict[str, int] = {}
        allocation_sum: dict[str, int] = {}
        reversal_targets: dict[str, list[dict]] = {}
```

```

reversals_unresolved: list[dict] = []
# payment -> ordered list of (bill_record_id, allocated_amount)
payment_allocations: dict[str, list[tuple[str, int]]] = {}

for e in events:
    if e.get("currency") not in ("", "USD"):
        self.conflicts.append(AmountConflict(
            "NON_USD_EVENT", f"event {e['event_id']} currency {e.get('currency')}
excluded",
            [e["event_id"]]))
        continue
    if e["event_type"] == "PAYMENT_POSTED":
        amt = e.get("amount_minor") or 0
        if amt > 0:
            payment_amount[e["event_id"]] = amt
    elif e["event_type"] == "PAYMENT_REVERSED":
        rev = (e.get("reverses_event_id") or "").strip()
        if rev:
            reversal_targets.setdefault(rev, []).append(e)
        else:
            reversals_unresolved.append(e)

for a in allocations:
    if a.get("currency") != "USD":
        self.conflicts.append(AmountConflict(
            "NON_USD_ALLOCATION", f"allocation {a['allocation_id']} excluded",
            [a["allocation_id"]]))
        continue
    peid = a["payment_event_id"]
    amt = a.get("allocated_amount_minor") or 0
    allocation_sum[peid] = allocation_sum.get(peid, 0) + amt
    bid = (a.get("bill_record_id") or "").strip()
    if bid:
        payment_allocations.setdefault(peid, []).append((bid, amt))

# over-allocation blocks the transaction
blocked = [peid for peid, amt in payment_amount.items()
            if allocation_sum.get(peid, 0) > amt]
for peid in blocked:
    self.conflicts.append(AmountConflict(
        "OVER_ALLOCATION",
        f"payment {peid} amount {payment_amount[peid]} "
        f"< allocation sum {allocation_sum[peid]}",
        [peid]))
    payment_amount.pop(peid, None)
    allocation_sum.pop(peid, None)

def lin_of_bill(bid: str) -> str:
    return lineage_of_record.get(bid, "UNLINKED")

def main_lineage(e: dict, allocs_list: list[tuple[str, int]]) -> str:
    """The lineage the unallocated remainder attaches to."""
    if e.get("bill_record_id"):
        return lin_of_bill(e["bill_record_id"])
    lins = {lin_of_bill(bid) for bid, _ in allocs_list}
    if len(lins) == 1:
        return next(iter(lins))
    return "UNLINKED"

```

```

def _parts(amt: int, allocs_list: list[tuple[str, int]],
           main_lin: str) -> list[tuple[str, int]]:
    """Attribution split of a payment amount across lineages."""
    parts: list[tuple[str, int]] = []
    for bid, a_amt in allocs_list:
        parts.append((lin_of_bill(bid), a_amt))
    unalloc = amt - sum(a for _, a in allocs_list)
    if unalloc > 0:
        parts.append((main_lin, unalloc))
    if not parts:
        parts.append((main_lin, amt))
    return parts

def _distribute(total: int, parts: list[tuple[str, int]]) -> list[tuple[str,
int]]:
    """Split `total` across parts in proportion to their amounts
    (integer floor, remainder to the last part)."""
    base = sum(a for _, a in parts)
    if base <= 0 or total <= 0:
        return [(lin, 0) for lin, _ in parts]
    used = 0
    out_rows: list[tuple[str, int]] = []
    for i, (lin, a) in enumerate(parts):
        share = total * a // base if i < len(parts) - 1 else total - used
        used += share
        out_rows.append((lin, share))
    return out_rows

# gross posted, unallocated and reversals per lineage
payment_reversed: dict[str, int] = {}
for e in events:
    if e["event_type"] != "PAYMENT_POSTED":
        continue
    peid = e["event_id"]
    amt = payment_amount.get(peid, 0)
    if amt <= 0:
        continue
    allocs_list = payment_allocations.get(peid, [])
    main_lin = main_lineage(e, allocs_list)
    parts = _parts(amt, allocs_list, main_lin)
    for lin, share in _distribute(amt, parts):
        acc = out.setdefault(lin, LineageAmounts(lineage_id=lin))
        acc.gross_posted += share
    unalloc_amt = amt - sum(a for _, a in allocs_list)
    if unalloc_amt > 0:
        acc = out.setdefault(main_lin, LineageAmounts(lineage_id=main_lin))
        acc.unallocated += unalloc_amt
    revs = reversal_targets.get(peid, [])
    for r in revs:
        r_amt = r.get("amount_minor") or 0
        # per-payment reversal cap: each payment's reversals can never
        # exceed that payment's own posted amount
        remaining = max(0, amt - payment_reversed.get(peid, 0))
        if remaining <= 0:
            break
        take = min(r_amt, remaining)
        payment_reversed[peid] = payment_reversed.get(peid, 0) + take

```

```
        for lin, share in _distribute(take, parts):
            acc = out.setdefault(lin, LineageAmounts(lineage_id=lin))
            acc.reversed += share

    for r in reversals_unresolved:
        self.conflicts.append(AmountConflict(
            "UNRESOLVED_REVERSAL",
            f"reversal {r['event_id']} has no target; not deducted anywhere",
            [r["event_id"]]))

    for acc in out.values():
        acc.net_observed_posted = acc.gross_posted - acc.reversed
    return out
```